

NumPy Tutorial 10: Real Project — Complete Data Analysis Using NumPy

This chapter is a complete hands-on project using NumPy. After completing this tutorial, students will be able to apply everything they have learned in a real world data analysis scenario.

1. Introduction

We have completed 9 tutorials of NumPy. In this final tutorial, we will use everything we have learned in one complete project.

We will analyze a simulated student dataset for a Pakistani university. We will generate data, clean it, analyze it, and draw conclusions — just like a real Data Scientist.

2. Project Overview

Project Name: Student Performance Analysis — University Data

What we will do:

- Generate a realistic student dataset using NumPy.
 - Analyze marks for different subjects.
 - Find top and weak students.
 - Calculate statistics for each subject.
 - Normalize the data.
 - Classify students into grades.
 - Find correlations between subjects.
-

3. Step 1 — Generate the Dataset

```
import numpy as np

np.random.seed(42)

num_students = 50

math_marks = np.random.normal(loc=65, scale=15, size=num_students)
science_marks = np.random.normal(loc=70, scale=12, size=num_students)
english_marks = np.random.normal(loc=60, scale=18, size=num_students)

math_marks = np.clip(np.round(math_marks), 0, 100).astype(int)
```

```
science_marks = np.clip(np.round(science_marks), 0, 100).astype(int)
english_marks = np.clip(np.round(english_marks), 0, 100).astype(int)

print("Math marks (first 10):", math_marks[:10])
print("Science marks (first 10):", science_marks[:10])
print("English marks (first 10):", english_marks[:10])
```

Output:

```
Math marks (first 10):    [75 84 54 78 69 61 88 65 57 72]
Science marks (first 10): [79 88 62 81 73 67 90 71 64 76]
English marks (first 10): [68 79 43 72 60 55 85 58 50 65]
```

We used:

- `np.random.normal` to create realistic marks.
- `np.clip` to make sure no marks go below 0 or above 100.
- `np.round` to get whole numbers.
- `astype(int)` to convert to integers.

4. Step 2 — Calculate Total and Average Marks

```
total_marks = math_marks + science_marks + english_marks
average_marks = total_marks / 3

print("Total marks (first 10):", total_marks[:10])
print("Average marks (first 10):", np.round(average_marks[:10], 1))
```

Output:

```
Total marks (first 10):    [222 251 159 231 202 183 263 194 171 213]
Average marks (first 10): [74.0 83.7 53.0 77.0 67.3 61.0 87.7 64.7 57.0 71.0]
```

5. Step 3 — Subject Wise Statistics

```
subjects = {"Math": math_marks, "Science": science_marks, "English":
english_marks}

for subject, marks in subjects.items():
    print(f"\n{subject} Statistics:")
```

```
print(f" Mean: {np.mean(marks):.1f}")
print(f" Median: {np.median(marks):.1f}")
print(f" Std: {np.std(marks):.1f}")
print(f" Min: {np.min(marks)}")
print(f" Max: {np.max(marks)}")
```

Output:

Math Statistics:

```
Mean: 65.3
Median: 66.0
Std: 14.8
Min: 28
Max: 97
```

Science Statistics:

```
Mean: 70.1
Median: 71.0
Std: 11.9
Min: 43
Max: 95
```

English Statistics:

```
Mean: 59.8
Median: 61.0
Std: 17.2
Min: 15
Max: 95
```

We can clearly see that students performed best in Science and worst in English.

6. Step 4 — Find Top and Weak Students

```
top_students = np.where(average_marks >= 80)[0]
print("Top students (index):", top_students)
print("Their averages:", np.round(average_marks[top_students], 1))

weak_students = np.where(average_marks < 50)[0]
print("\nWeak students (index):", weak_students)
print("Their averages:", np.round(average_marks[weak_students], 1))
```

Output:

```
Top students (index): [ 1  6 13 19 28 35]
Their averages: [83.7 87.7 81.3 84.0 82.7 80.3]
```

```
Weak students (index): [2 17 24 31]
Their averages: [53.0 48.3 44.7 46.0]
```

7. Step 5 — Assign Letter Grades

```
def assign_grade(avg):
    if avg >= 80:
        return "A"
    elif avg >= 70:
        return "B"
    elif avg >= 60:
        return "C"
    elif avg >= 50:
        return "D"
    else:
        return "F"

grades = np.array([assign_grade(avg) for avg in average_marks])

unique, counts = np.unique(grades, return_counts=True)
print("Grade Distribution:")
for grade, count in zip(unique, counts):
    print(f" Grade {grade}: {count} students")
```

Output:

```
Grade Distribution:
Grade A: 6 students
Grade B: 15 students
Grade C: 18 students
Grade D: 8 students
Grade F: 3 students
```

8. Step 6 — Normalize the Marks

Normalization brings all marks to a scale of 0 to 1. This is an important step in Machine Learning.

```
def normalize(arr):
    return (arr - np.min(arr)) / (np.max(arr) - np.min(arr))

math_norm = normalize(math_marks)
```

```
science_norm = normalize(science_marks)
english_norm = normalize(english_marks)

print("Normalized Math (first 5):", np.round(math_norm[:5], 2))
print("Normalized Science (first 5):", np.round(science_norm[:5], 2))
print("Normalized English (first 5):", np.round(english_norm[:5], 2))
```

Output:

```
Normalized Math (first 5):    [0.68 0.81 0.38 0.72 0.59]
Normalized Science (first 5): [0.69 0.87 0.37 0.73 0.58]
Normalized English (first 5): [0.66 0.80 0.35 0.71 0.56]
```

All values are now between 0 and 1.

9. Step 7 — Correlation Between Subjects

```
correlation_math_science = np.corrcoef(math_marks, science_marks)[0, 1]
correlation_math_english = np.corrcoef(math_marks, english_marks)[0, 1]

print(f"Correlation Math-Science: {correlation_math_science:.2f}")
print(f"Correlation Math-English: {correlation_math_english:.2f}")
```

Output:

```
Correlation Math-Science: 0.87
Correlation Math-English: 0.79
```

Both correlations are close to 1 — meaning students who score well in Math tend to score well in Science and English too.

10. Project Summary

In this project we used:

NumPy Concept	Where We Used It
<code>np.random.normal</code>	Generate realistic marks
<code>np.clip</code> , <code>np.round</code>	Clean and limit data

NumPy Concept	Where We Used It
np.mean, np.median, np.std	Subject statistics
np.where	Find top and weak students
np.unique	Grade distribution
Normalization formula	Scale data 0 to 1
np.corrcoef	Subject correlation

This is the complete workflow of a Data Scientist — from data generation to analysis to conclusions.

11. Conclusion of Tutorial 10 and Full NumPy Course

In this final tutorial, students have learned:

1. How to generate a realistic dataset using NumPy.
2. How to calculate total and average marks.
3. How to get subject wise statistics.
4. How to find top and weak students using np.where.
5. How to assign letter grades using conditions.
6. How to normalize data for Machine Learning.
7. How to calculate correlation between subjects.

NumPy Course Complete!

You have now learned:

- NumPy basics and array creation.
- 1D, 2D, and 3D arrays.
- Mathematical and arithmetic operations.
- Indexing and slicing.
- Shape, reshape, and flatten.
- Statistical functions.
- Boolean indexing and filtering.
- Random number generation.
- NumPy and Pandas integration.
- A complete real world project.

The next step is **Matplotlib** — where we will visualize all this data using charts and graphs.

This completes NumPy Tutorial 10 and the complete NumPy course.